

PROFESSIONAL SUMMARY

Distributed LLM inference and scheduling engineer with five years of cloud-native AI/HPC experience and nearly three years in production serving. Leads Kubernetes-native runtime/control-plane work across worker lifecycle and model loading, KV-event propagation and routing, P/D capacity planning, and scaling guardrails; also delivered production batch inference and model gateway systems. Primary languages: Go and Python, with Rust routing experience.

EXPERIENCE

Zhejiang Lab · High-Performance Computing Facility Research Center

Senior Research Specialist | LLM Serving / Model-as-a-Service (MaaS) Platform

2024 - Present

- Led a distributed inference runtime/control plane spanning worker bootstrap, fast model loading, KV-block events, KV-aware routing, adaptive P/D capacity planning, and Kubernetes lifecycle orchestration; modeled process, model, event-plane, and traffic readiness as distinct states.
- Designed and implemented an auditable adaptive P/D Planner that combines request rate, ISL/OSL, prefill cached/compute tokens, KV hit rate, queue/running state, and TTFT/TPOT; mapped live request shapes to Profiler capacity profiles to produce separate prefill/decode targets and a bottleneck-driven scale intent.
- Designed and implemented an OpenAI Batch API-compatible execution platform covering file and batch lifecycles, task sharding, worker execution, Kubernetes Lease-based leader election, tenant concurrency, recovery, and result/error writeback; supported production data-distillation and batch-generation workloads at ~10M requests/day with a >99% request success rate.
- Integrated SGLang HTTP/gRPC P/D routing, multi-tenant admission, and end-to-end tracing; load-tested a heterogeneous multi-backend SGLang routing path with ~300K requests and 9.75B tokens, achieving a ~99.9% request success rate; traced production failures across request ID → Gateway → Router → Worker.

Zhejiang Lab · Intelligent Supercomputing Research Center

Engineering Specialist | Cloud-Native AI / HPC Platform

2021 - 2023

- Customized Kubernetes device-plugin and resource-management paths for heterogeneous accelerator discovery, node resource reporting, container-runtime validation, device-level allocation, and fault isolation.
- Adapted PyTorch/models and distributed job environments to Linux distributions and heterogeneous accelerator stacks, establishing the platform foundation for later GPU/XPU serving and scheduling work.

SELECTED SYSTEMS

Distributed Inference Runtime & KV-Aware Routing

Worker Bootstrap · ModelExpress · Event Plane

- Extended worker startup from process launch into a runtime initialization contract: attached engine lifecycle to DistributedRuntime; registered request-plane endpoints and Model Deployment Cards separately with WorkerType, dependency DNF, KV capacity, and topology metadata; integrated the ModelExpress loader for P2P/NIXL-RDMA weight distribution during scale-out.
- Initialized a KvEventPublisher per DP rank during bootstrap, normalized native SGLang/vLLM BlockStored, BlockRemoved, and AllBlocksCleared events from ZMQ, and published them to the event plane for the worker KV-block index.
- Combined prefix KV overlap, worker load, and topology constraints for routing; fell back to a token-level approximate index when events were unavailable, while controllers coordinated Pod, Endpoint, model, event-plane, and Router readiness.

SELECTED SYSTEMS (CONTINUED)

Adaptive P/D Planner and Scaling Guardrails	Capacity Profile · ScaleIntent · Guardrails
- Derived request rate, input/output tokens, prefill compute/cached tokens, and KV hit rate from rolling windows; mapped live request shapes onto profiled prefill/decode capacity curves with interpolation and conservative fallback. - Computed prefill/decode token demand, safe per-worker throughput, and phase targets independently, then used the bottleneck as the aggregate target; preserved request shape, selected profile points, utilization, and blockers for shadow validation and decision audit. - Decoupled Planner, ScaleIntent Controller, ScalingAdapter, IGD/ISI, and Scheduler; applied desired-vs-ready pending gates, one-step scaling, stable scale-down windows, cooldown, and stale-intent checks to prevent oscillation and concurrent writes.	
Multi-Tenant Batch Inference Platform	OpenAI Batch API · Scheduling · Recovery
- Modeled batches as durable, recoverable asynchronous jobs rather than bulk request scripts: the database owns task truth while schedulers/workers claim, shard, execute, renew, and finalize work. - Enabled multi-replica operation with Kubernetes Lease-based leader election, database-level claims, tenant/model concurrency budgets, and request-level error isolation, preventing duplicate execution and limiting offline impact on online serving. - Implemented streaming JSONL processing, cancellation, expiry cleanup, output/error files, and failure-tolerant finalization from API contract through production monitoring.	
Inference Router and Multi-Tenant Admission	P/D Routing · Priority Admission · Tracing
- Implemented P0-P3 admission in Rust with global/tier semaphores, strict priority scheduling, per-tenant round-robin, bounded queues, and wait timeouts; held permits through streaming response-body completion to avoid releasing capacity early. - Integrated SGLang HTTP/gRPC P/D routing, prefill/decode bootstrap and transfer context, plus request-ID and trace-context propagation; validated the multi-backend path with QPS, TTFT, TPOT, and KV-hit telemetry.	

CORE TECHNOLOGIES

Inference Runtime & Routing	SGLang, vLLM, NVIDIA Dynamo, AIBrix, ModelExpress, NIXL/RDMA, KV event plane, KV-aware routing, P/D disaggregation
Scheduling & Control Plane	Go, Kubernetes, CRDs/controllers, capacity profiles, scale intents, reconciliation, readiness, finalizers, leader election, Helm, ArgoCD
Gateway & Async Serving	Python, Rust, OpenAI-compatible APIs, Batch API, LiteLLM, Higress, Redis, MySQL, multi-tenant concurrency, failure recovery
Observability & Performance	Prometheus, request tracing, QPS/TTFT/TPOT, ISL/OSL, KV hit rate, throughput curves, distributed-systems debugging

EDUCATION

Arizona State University	May 2019 - Dec 2020
Master of Computer Science, GPA 3.97 / 4.00	
Arizona State University	Aug 2015 - May 2019
B.S. in Computer Science	

INTELLECTUAL PROPERTY

Co-inventor on two granted invention patents, including an elastic cloud model-inference serving system; contributor to two authorized software copyrights.